

Token Sparse Attention: Efficient Long-Context Inference with Interleaved Token Selection

Dongwon Jo¹ Beomseok Kang¹ Jiwon Song¹ Jae-Joon Kim¹
<https://github.com/dongwonjo/Token-Sparse-Attention>

Abstract

The quadratic complexity of attention remains the central bottleneck in long-context inference for large language models. Prior acceleration methods either sparsify the attention map with structured patterns or permanently evict tokens at specific layers, which can retain irrelevant tokens or rely on irreversible early decisions despite the layer-/head-wise dynamics of token importance. In this paper, we propose Token Sparse Attention, a lightweight and dynamic token-level sparsification mechanism that compresses per-head Q, K, V to a reduced token set during attention and then decompresses the output back to the original sequence, enabling token information to be reconsidered in subsequent layers. Furthermore, Token Sparse Attention exposes a new design point at the intersection of token selection and sparse attention. Our approach is fully compatible with dense attention implementations, including Flash Attention, and can be seamlessly composed with existing sparse attention kernels. Experimental results show that Token Sparse Attention consistently improves accuracy–latency trade-off, achieving up to $\times 3.23$ attention speedup at 128K context with less than 1% accuracy degradation. These results demonstrate that dynamic and interleaved token-level sparsification is a complementary and effective strategy for scalable long-context inference.

1. Introduction

The capability to process long contexts has become a defining feature of modern Large Language Models

¹Department of Electrical and Computer Engineering, Seoul National University, Seoul, South Korea. Correspondence to: Beomseok Kang, Jae-Joon Kim <beomseok, kimjaejuon@snu.ac.kr>.

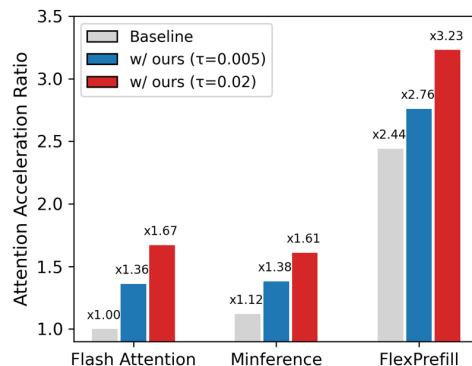


Figure 1. Speedups with Token Sparse Attention. Attention acceleration ratios obtained by applying the proposed Token Sparse Attention (ours) to existing attention acceleration methods. τ denotes the sparsity level (*i.e.*, higher the sparser).

(LLMs) (Achiam et al., 2023; Comanici et al., 2025; Anthropic, 2024), enabling applications ranging from long document summarization to multi-turn reasoning and code generation (Yi et al., 2024; Laban et al., 2023; Rando et al., 2025). However, as context lengths (L) increase, the complexity of attention mechanism grows quadratically ($O(L^2)$) during the prefill stage, creating a fundamental bottleneck in inference. While hardware-aware optimizations such as FlashAttention (Dao, 2023) significantly reduce memory I/O overhead, the inherent quadratic complexity remains the same, necessitating algorithmic innovations to scale practical context lengths beyond 100K.

Recent state-of-the-art (SOTA) methods for accelerating prefill predominantly rely on *sparse attention* (Lai et al., 2025; Jiang et al., 2024a), where computations for low-importance regions in the attention map ($QK^\top$) are bypassed. While these methods approximate attention weights effectively, they typically impose block-level sparsity to stay compatible with the execution patterns of attention kernels. As a result, less informative tokens may still be preserved when grouped with salient tokens in the same block, potentially leading to sub-optimal sparsity. A few studies have explored token-level sparsity during prefill (Jo et al., 2025; Shi et al., 2024). These methods identify a subset of important tokens in the early layers and permanently discard the rest in deeper layers. However, once tokens are evicted, they cannot be reconsidered even if they become significant in

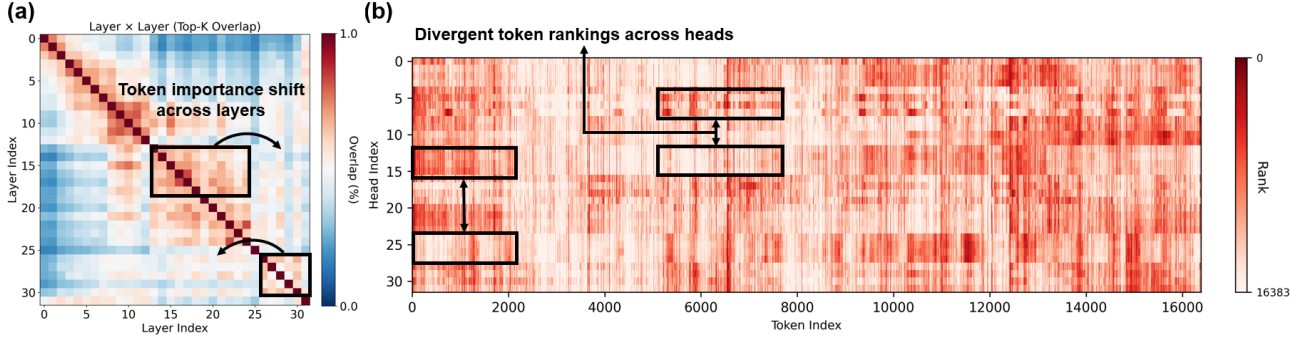


Figure 2. **Dynamics of Token Importance.** (a) Layer-wise overlap of top-k important tokens, showing that token importance shifts significantly across layers. (b) Head-wise token importance rankings within the same layer, illustrating that different attention heads prioritize different subsets of tokens.

subsequent layers. This naturally enforces the same set of tokens to persist across layers and fails to capture more selective patterns such as layer- or/and head-specific shifts in token importance. Consequently, eviction-based approaches often exhibit weaker accuracy-speedup trade-offs, falling short of fully leveraging the benefits of operating at a finer (token-level) granularity.

In this paper, we introduce **Token Sparse Attention**, a novel token-level attention mechanism for accelerating prefill in long-context inference. At each head, the method selects a small subset of informative tokens ($L' < L$) and performs attention efficiently within the resulting compressed space ($\mathbb{R}^{L' \times L'}$), allowing the selected subset to differ across heads. However, naïvely applying this selection prevents the remaining tokens ($L - L'$) from being re-visited in subsequent layers, inevitably evicting them. To address this issue, we interleave the attention output ($\mathbb{R}^{L' \times d}$) back into the original sequence dimension ($\mathbb{R}^{L \times d}$). This “Compress and then Decompress” design enables each head to repeatedly select its own significant tokens from the full sequence (L), reaping the benefits of head-specific token selection. In addition, because the full sequence dimension is restored ($L' \rightarrow L$) at every layer, our design naturally supports layer-wise adaptive sparsity budgets. Consequently, our method dynamically determines the sparsity level on-the-fly during inference, providing a flexibility that eviction-based approaches inherently lack.

While improving performance on its own, Token Sparse Attention’s core strength lies in being complementary to existing sparse attention methods rather than replacing them. This arises from its strong compatibility: irrelevant tokens can be pruned prior to any sparse computations, *e.g.*, block-sparse or Λ -shape. Because our method essentially performs dense attention within the compressed space ($\mathbb{R}^{L' \times L'}$), it integrates seamlessly with implementations like FlashAttention and sparse attention kernels. As a result, we demonstrate heterogeneous granularity (see Figure 1) as an effective new strategy for enhancing sparse attention, *e.g.*, com-

binning FlexPrefill (Lai et al., 2025) (block-sparse) with ours (token-sparse) reaches 87.3% accuracy with $\times 2.8$ speedup than FlashAttention, whereas standard FlexPrefill achieves the same accuracy (87.3%) but $\times 2.4$ speedup (see Table 1).

2. Method

2.1. Motivation

Prior token-sparse techniques (Jo et al., 2025; Shi et al., 2024) generally reduce the computational cost of attention by selecting a subset of tokens in early layers and restricting computation to only those tokens in subsequent layers. However, this strategy implicitly assumes that token importance estimated in early layers will remain stable throughout the model during prefill, which rarely holds in practice. To illustrate this, we analyze token-importance dynamics during prefill for a LLaMA-3.1-8B-Instruct model (Dubey et al., 2024), as shown in Figure 2.

Layer-wise Token Importance. Figure 2(a) reports the layer-wise overlap of important tokens, where we select the top 1% tokens at each layer and measure their pairwise overlap across layers. Although adjacent layers share a moderate fraction of important tokens, the overlap decreases rapidly as the layer distance increases. This indicates that token importance shifts substantially across layers as attention focus evolves during prefill. As a result, permanently removing tokens based on early-layer selection can prematurely eliminate candidates for subsequent layers that might later become relevant.

Head-wise Token Importance. Figure 2(b) further shows token-importance variation across attention heads at layer 18. Different heads exhibit distinct token-ranking patterns, suggesting that token relevance is not only layer-dependent but also head-specific. This heterogeneity is a fundamental property of multi-head attention, where heads specialize in capturing different contextual relationships. Thus, enforcing a coarse, layer-level eviction policy forces all heads to share a unified token set, inevitably discarding tokens that

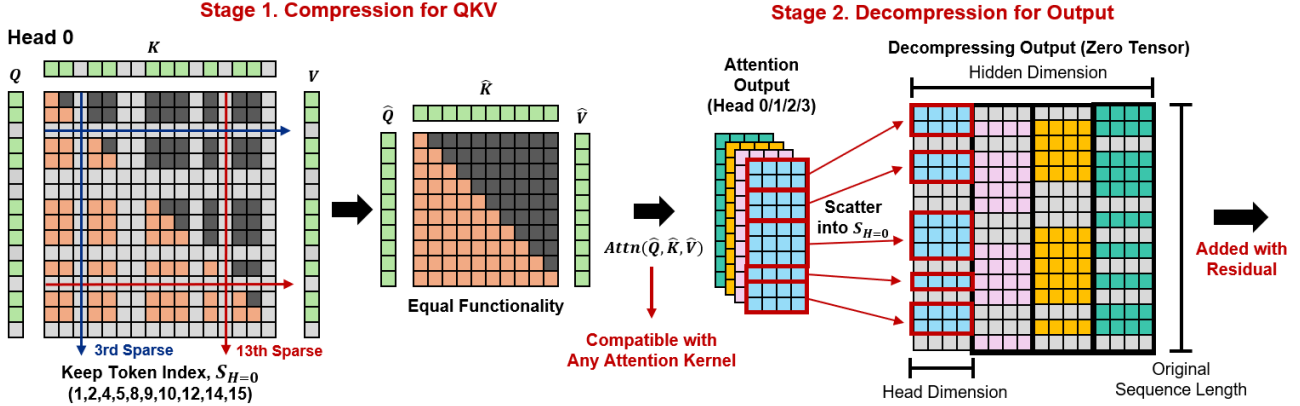


Figure 3. **Overview of the proposed Token Sparse Attention.** Stage 1 compresses Q , K , and V by selecting a per-head token subset $S_{H=0}$, yielding compact $\hat{Q}$, $\hat{K}$, and $\hat{V}$ that remain compatible with standard attention kernels. Stage 2 performs attention on the compressed tensors and scatters the resulting outputs back into the full sequence layout before adding the residual connection.

individual heads may find essential.

These observations indicate that token selection should retain flexibility across both layers and heads, rather than permanently committing to early-layer decisions. Motivated by this, Token Sparse Attention is designed to accommodate these dynamics through interleaved token selection, which restores the full sequence dimension after each compressed attention step. This reversible design reduces unnecessary computation while preserving future selection space.

2.2. Token Sparse Attention

Motivated by the observations in Section 2.1, we aim to selectively skip irrelevant tokens during attention computation without permanently removing them from the sequence. As illustrated in Figure 3, this process consists of two primary stages which are compression for query-key-value tensors and decompression for the attention output. These stages ensure that the computational benefits of sparsity are realized while maintaining the structural integrity of the original sequence.

Stage 1: Compression for QKV. In the compression stage, each attention head h selects a subset of token indices $S_{H=h}$, yielding a reduced sequence length $L' \ll L$. Using these indices, we gather the corresponding rows from the original Q , K , and V tensors to construct compressed tensors $\hat{Q}$, $\hat{K}$, and $\hat{V}$. Importantly, this selection is performed independently per head, allowing different heads to attend to different subsets of tokens. This design directly addresses the head-wise heterogeneity observed in Figure 2(b). A major advantage of this compression strategy is that the resulting tensors $\hat{Q}$, $\hat{K}$, $\hat{V}$ remain dense and contiguous in memory. This allows us to utilize highly optimized hardware-aware kernels such as FlashAttention or other specialized sparse attention implementations without any modification. The attention operation is performed on the compressed ten-

sors to produce a reduced output tensor $\hat{O}$, which contains the context-aware representations for the selected tokens, thereby reducing the quadratic attention cost from $O(L^2d)$ to $O(L'^2d)$.

Stage 2: Decompression for Attention Output. In the decompression stage following attention computation, the compressed output $\hat{O}$ is scattered back into a zero-initialized tensor of shape $\mathbb{R}^{L \times d}$ using the head-specific index set $S_{H=h}$. This operation ensures that the output dimensions match the original input, preventing any dimension mismatch issues in subsequent layers. The unselected positions in the output tensor remain zero, which is functionally equivalent to applying a hard mask to those tokens in the attention map. Finally, the restored attention output is added to the residual connection. This step is critical because the residual connection preserves the information of the unselected tokens from the previous layer.

As a result, Token Sparse Attention achieves dynamic token-level sparsification without sacrificing the expressive flexibility of dense attention. The model can selectively ignore irrelevant context to reduce computation, yet continuously re-evaluate token importance across layers and heads. In the next subsection, we describe how token importance is estimated and how the per-head token subsets S_H are dynamically determined under a sparsity budget.

2.3. Dynamic Token Coverage

Token Sparse Attention adaptively selects the sparsity budget at inference time, which involves two key decisions: (1) determining how many tokens to retain and (2) identifying which tokens to keep for each attention head. To address this, we introduce **Dynamic Token Coverage**, a token-selection policy applied before the compression stage of Token Sparse Attention. The full procedure is summarized in Algorithm 1.

We begin by estimating token importance independently

Algorithm 1 Dynamic Token Coverage and Index Search**Require:** Q, K , Token Coverage τ , Sequence Length L **Ensure:** Head-wise selected token indices $\{S_h\}_{h=1}^H$

```

1: // Compute attention scores using recent queries
2:  $\hat{A} \leftarrow \text{softmax}\left(Q_{[-last\_q:]} K^\top / \sqrt{d}\right)$ 
3: // Compute token-level scores per head
4:  $s_h \leftarrow \text{pooling}(\text{sum}_{\text{vertical}}(\hat{A}))$ 
5: // Aggregate and normalize token scores across heads
6:  $s_l \leftarrow (\sum_{h=1}^H s_h) / (\sum_{t=1}^L \sum_{h=1}^H s_h[t])$ 
7: // Sort tokens by ascending importance
8:  $I \leftarrow \text{argsort}(s_l)$ 
9: // Determine the number of sparse token
10:  $k_{\text{sparse}} \leftarrow \arg \min_{k \in \{0, \dots, L\}} \left\{ \sum_{j=1}^k s_l[I[j]] \geq \tau \right\}$ 
11:  $k_{\text{keep}} \leftarrow L - k_{\text{sparse}}$ 
12: // Head-wise top- $k_{\text{keep}}$  token selection
13:  $\{S_h\}_{h=1}^H \leftarrow \{\text{TopK}(s_h, k_{\text{keep}})\}_{h=1}^H$ 
14: return  $\{S_h\}_{h=1}^H$ 

```

for each attention head. For a given head h , we compute a lightweight proxy of the attention map $\hat{A}$ by attending a small set of recent queries to all keys. The importance score $s_h[t]$ for the t -th token is derived by summing the attention weights along the vertical axis (*i.e.*, the sequence length dimension of queries). We implement this scoring step using a custom hardware-efficient kernel developed in Triton (Tillet et al., 2019) that minimizes memory I/O overhead by fusing the score calculation.

How Many Tokens to Retain. Given the head-wise token scores, we aggregate them into a layer-level importance distribution by summing scores across heads and normalizing over the sequence dimension. This aggregated score captures the overall contribution of each token at the current layer and serves as the basis for determining the sparsity budget.

Instead of selecting tokens in descending order of importance, we sort tokens by ascending estimated importance and identify the minimal set of least important tokens whose cumulative mass exceeds a predefined coverage threshold τ . A key premise of our approach is that long-context attention accumulates attention noise, manifested as a long tail of tokens with negligible cumulative contribution. We interpret pruning this tail as a form of structural regularization that reduces distraction from irrelevant context.

Which Tokens to Keep. Once the layer-wise budget is determined, we perform the final token selection independently for each attention head by selecting the top- k_{keep} tokens according to s_h . This design satisfies the heterogeneity requirement discussed in Section 2.1, allowing each head to attend to distinct semantic features while utilizing the allocated compute budget optimally.

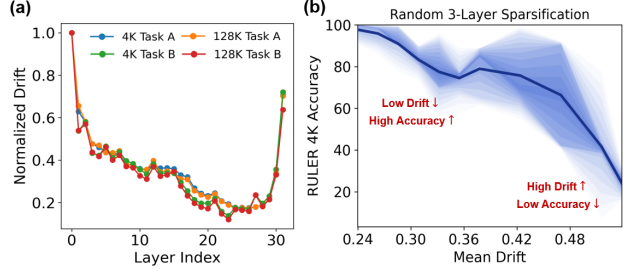


Figure 4. **Sparse Layer Selection.** (a) Layer-wise normalized drift measured across different tasks and context lengths. Task A and B correspond to a retrieval task and a summarization task, respectively. (b) Relationship between accuracy and drift under random 3-layer sparsification over 200 runs.

2.4. Sparse Layer Selection

Token Sparse Attention and Dynamic Token Coverage determine how tokens are selected within each layer. However, we observe that applying Token Sparse Attention across all layers leads to substantial performance degradation. This raises an orthogonal question: which layers can accommodate token-level sparsification with minimal impact on model behavior?

To identify layers where token representations are sufficiently stable for sparsification, we introduce a metric called **Inter-Layer Representation Drift**, which measures the relative change in a token’s representation by comparing the L2 norms of its input and output hidden states. For a given layer ℓ , the drift R_ℓ is defined as:

$$R_\ell = \mathbb{E}_t \left[\frac{\|h_{\ell+1,t} - h_{\ell,t}\|_2}{\|h_{\ell,t}\|_2 + \epsilon} \right], \quad (1)$$

where $h_{\ell,t}$ denotes the hidden state (*i.e.*, layer input) of token t at layer ℓ . A lower drift value indicates smaller representational changes across layers, suggesting larger stability in token representations. As illustrated in Figure 4(a), the drift exhibits a consistent pattern across varying tasks and context lengths on LLaMA-3.1-8B-Instruct.

To investigate whether the drift is predictive of sparsification robustness, we conduct a controlled experiment shown in Figure 4(b). Specifically, we randomly sample three layers from the model and apply Token Sparse Attention only to these layers with token coverage of 0.99, while keeping other layers dense. For each sampled triplet, we compute the mean normalized drift of the selected layers and evaluate the resulting accuracy on the RULER benchmark (4K-length). This process is repeated over 200 runs. The results reveal a clear correlation: layer subsets with lower average drift tend to exhibit higher accuracy, validating R_ℓ as a reliable indicator for sparsity robustness. Based on this observation, we use representation drift as a criterion to select layers for token-level sparsification.

We define a normalized drift rank for each layer and select

Table 1. Complementary Gains from Token Sparse Attention on RULER. Adding Token Sparse Attention on top of FlashAttention and existing sparse attention methods improves speed with marginal accuracy variation across all context lengths and models. Speedup denotes the 128K attention speedup relative to the FlashAttention baseline.

Method	4K	8K	16K	32K	64K	128K	Avg.	Speedup
LLaMA-3.1-8B-Instruct								
Flash-Attention	95.82	92.77	91.02	84.87	83.43	74.15	87.01	$\times 1.00$
<i>w/ Token Sparse</i>	96.06	92.90	91.82	84.81	82.83	73.68	87.02	$\times 1.36$
Minference	93.46	92.29	91.01	85.34	83.19	73.63	86.49	$\times 1.12$
<i>w/ Token Sparse</i>	93.05	92.00	91.03	85.10	82.92	72.18	86.05	$\times 1.38$
FlexPrefill	95.48	92.71	91.40	87.20	83.05	73.75	87.27	$\times 2.44$
<i>w/ Token Sparse</i>	95.33	92.47	91.49	87.68	83.07	73.58	87.27	$\times 2.76$
Mistral-Nemo-12B-Instruct								
Flash-Attention	95.19	92.29	85.60	64.86	47.98	19.68	67.60	$\times 1.00$
<i>w/ Token Sparse</i>	95.07	92.10	85.97	63.67	47.97	19.44	67.37	$\times 1.22$
Minference	92.52	91.02	84.29	65.18	46.52	19.00	66.42	$\times 1.13$
<i>w/ Token Sparse</i>	93.01	91.36	84.31	64.70	46.67	18.68	66.46	$\times 1.28$
FlexPrefill	94.79	93.13	86.62	64.58	49.30	20.54	68.16	$\times 1.22$
<i>w/ Token Sparse</i>	94.84	93.31	86.14	64.89	48.70	19.58	67.91	$\times 1.33$

the subset of layers eligible for Token Sparse Attention as:

$$\hat{R}_\ell = \frac{1}{L} \sum_{k=1}^L \mathbf{1}[R_k \leq R_\ell], \quad \mathcal{L}_{\text{sparse}} = \{\ell \mid \hat{R}_\ell \leq \delta\} \quad (2)$$

In all our experiments, we set $\delta = 0.5$, applying our method strictly to the layers exhibiting the most stable representations. This layer selection is performed once as a preprocessing step for each model.

3. Experiments

Our experiments first demonstrate the complementary gains achieved when Token Sparse Attention is combined with other attention-acceleration techniques in Section 3.2. We further compare it against token-eviction methods in Section 3.4. A detailed analysis of the accuracy-efficiency trade-off is provided in Section 3.3.

3.1. Experimental Setup

Models and Datasets. We conduct experiments on LLaMA-3.1-8B-Instruct (Dubey et al., 2024) and Mistral-Nemo-12B-Instruct (Mistral AI Team, 2025). For evaluation, we primarily use RULER (Hsieh et al., 2024) and InfiniteBench (Zhang et al., 2024), two benchmarks designed to assess long-context understanding and retrieval performance. Evaluation on LongBench (Bai et al., 2023) and Needle-in-a-Haystack (Kamradt, 2023) is available in the Appendix A.2.

Implementation Details. Token Sparse Attention is integrated into FlashAttention without modifying the underlying

kernel, and the token-scoring step in Section 2.3 is implemented using a Triton kernel (Tillet et al., 2019). To balance efficiency and accuracy, we use a token-coverage parameter of $\tau=0.005$ for LLaMA and $\tau=0.008$ for Mistral across all experiments. All experiments are conducted on a single NVIDIA A100 80GB GPU.

Baselines. To validate our method’s compatibility, several attention acceleration baselines are considered. FlashAttention (Dao, 2023) serves as the dense attention baseline, providing highly optimized exact attention with improved memory efficiency. Minference (Jiang et al., 2024a) is a structured sparse attention method that applies predefined sparsity patterns to the attention map. We follow its official configuration, in which a Vertical-Slash sparsity pattern is applied uniformly across all attention heads. FlexPrefill (Lai et al., 2025) is a context-aware block-sparse attention method that dynamically prunes attention computation during prefill. Following the original paper, we set its hyperparameters to $\gamma=0.95$. To compare our method against token-eviction methods, PyramidInfer (Yang et al., 2024a), FastKV (Jo et al., 2025) and GemFilter (Shi et al., 2024) are primarily considered. For fair comparison, all methods are applied only during prefill, while decoding uses standard dense attention. Additional experiments are provided in Appendix A.5.

3.2. Accuracy Results

RULER. Table 1 reports the complementary performance of several attention-acceleration methods with and without Token Sparse Attention. Across all context lengths, To-

Table 2. Complementary Results from Token Sparse Attention on InfiniteBench.

Method	En.MC	En.QA	En.Dia	Retr.KV	Retr.N	Retr.P	Math.F	Code.D	Avg.
LLaMA-3.1-8B-Instruct									
Flash-Attention	64.19	27.64	19.00	55.20	98.47	97.80	24.00	20.56	50.86
w/ Token Sparse	65.07	28.01	17.00	55.00	98.47	97.46	25.71	20.30	50.88
Minference	67.25	26.80	18.50	51.60	97.46	97.80	22.29	19.54	50.16
w/ Token Sparse	65.94	27.61	18.50	49.60	97.46	97.80	21.43	19.29	49.70
FlexPrefill	68.12	27.37	11.00	50.80	99.15	96.95	22.57	20.30	49.53
w/ Token Sparse	67.25	27.60	14.00	46.80	98.98	96.27	22.86	20.05	49.23
Mistral-Nemo-12B-Instruct									
Flash-Attention	33.19	16.87	5.50	0.00	36.61	62.71	1.43	27.16	22.93
w/ Token Sparse	33.19	16.81	5.50	0.00	35.76	62.88	0.86	27.41	22.80
Minference	37.99	16.76	5.50	0.00	34.58	31.19	6.00	26.14	19.77
w/ Token Sparse	38.43	16.30	5.50	0.00	33.73	30.00	5.14	26.14	19.41
FlexPrefill	35.81	17.45	7.00	0.00	41.19	65.76	4.57	26.65	24.80
w/ Token Sparse	35.37	16.71	6.50	0.00	39.32	65.93	3.14	25.63	24.08

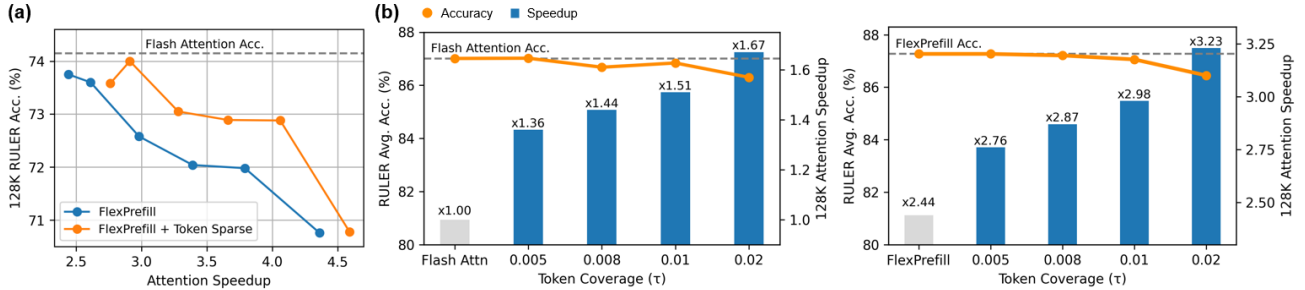


Figure 5. **Accuracy-Speedup Trade-offs.** (a) Accuracy-speedup Pareto frontier obtained by sweeping FlexPrefill hyperparameters, comparing with the Token Sparse Attention at token coverage of $\tau=0.005$. (b) Accuracy-speedup trade-off achieved by applying Token Sparse Attention with varying token coverage to FlashAttention (left) and FlexPrefill (right).

ken Sparse Attention largely preserves the accuracy of the underlying attention kernels across context lengths while improving the attention efficiency. For example, when applied to FlexPrefill on LLaMA-3.1-8B-Instruct, the average accuracy remains unchanged at 87.27%, matching the vanilla FlexPrefill result. A similar trend is observed when composing with FlashAttention and Minference. For Mistral-Nemo-12B-Instruct, the deviation introduced by Token Sparse Attention is consistently small, staying within 0.5% of the baseline. In terms of efficiency, Token Sparse Attention reliably increases the attention speedup at 128K across all methods, *e.g.*, for LLaMA-3.1-8B-Instruct, Minference improves from $\times 1.12$ to $\times 1.38$, and FlexPrefill from $\times 2.44$ to $\times 2.76$. Overall, Token Sparse Attention follows the baseline behavior closely, providing complementary efficiency gains with negligible impact on model accuracy.

InfiniteBench. Table 2 shows that Token Sparse Attention also preserves performance on InfiniteBench while remaining fully compatible with all baseline methods. For both LLaMA-3.1-8B-Instruct and Mistral-Nemo-12B-Instruct, augmenting Flash Attention, Minference, and FlexPrefill

with Token Sparse Attention leads to only marginal accuracy differences, with overall results closely matching those of the corresponding baselines. The efficiency gains follow the same trend observed in Table 1. These results highlight that Token Sparse Attention acts as a general acceleration mechanism, providing consistent inference efficiency improvements while maintaining baseline accuracy.

3.3. Efficiency Results

Accuracy-Speedup Trade-offs. We further analyze the computational benefits of Token Sparse Attention with respect to the accuracy-efficiency trade-offs. The attention speedup is estimated by the average attention latency measured across all layers. Figure 5(a) illustrates the Pareto frontier obtained by varying the hyperparameter γ , which represents the sparsity parameter, of FlexPrefill, a SOTA sparse attention (block-level) baseline. While aggressively tuning γ yields higher speedups, it often comes at the cost of rapid accuracy degradation. In contrast, applying Token Sparse Attention (with $\tau=0.005$) on top of FlexPrefill consistently pushes the Pareto frontier outward, achieving

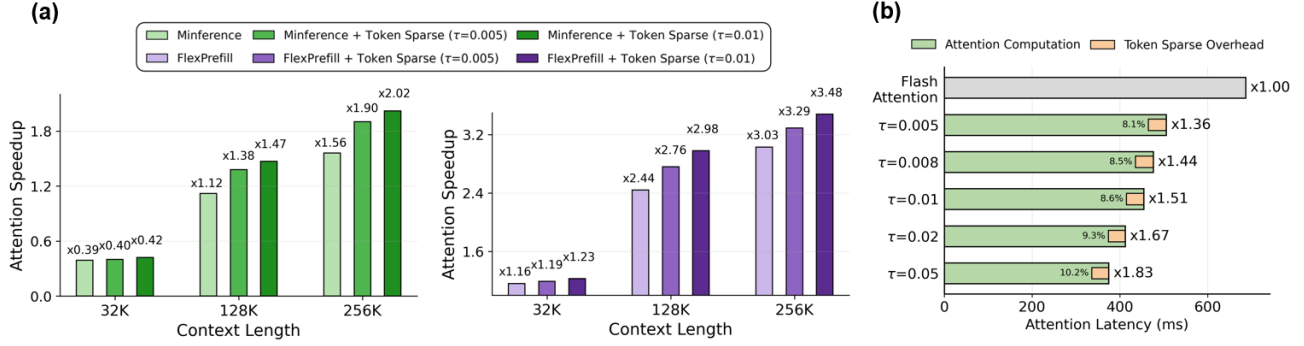


Figure 6. **Attention Speedup Details.** (a) Attention speedup across different context lengths. (b) Attention latency breakdown with Token Sparse Attention at 128K context, where the overhead includes token scoring and indexing, as well as QKV compression and attention output decompression.

Table 3. **Attention Map Sparsity.** Average attention map sparsity across different context lengths under different token coverage, τ .

Sparsity	4K	8K	16K	32K	64K	128K
$\tau=0.005$	17.00%	21.11%	26.61%	28.44%	34.07%	54.44%
$\tau=0.010$	28.02%	32.98%	39.55%	41.25%	47.32%	67.36%

superior speedups at comparable accuracy levels. This verifies that our method provides a complementary efficiency gain that cannot be achieved by simply adjusting hyperparameter.

Furthermore, we examine the effect of the token coverage parameter τ on performance. As shown in Figure 5(b), increasing τ enables more aggressive token sparsification, yielding higher attention speedups for both FlashAttention and FlexPrefill. Remarkably, this aggressive reduction does not compromise model performance. The accuracy degradation remains within 1% even at higher sparsity levels. This robustness suggests that Token Sparse Attention effectively targets and removes only the irrelevant tokens in the context, allowing users to flexibly trade off a small margin of accuracy for significant gains in inference speed.

Sparsity and Speedup across Sequence Lengths. Figure 6(a) shows that the attention speedup achieved by Token Sparse Attention increases consistently as the context length increases. While the gains are modest at shorter sequence lengths, substantially larger speedups are observed at 128K and 256K where attention computation dominates the overall latency. This behavior is explained by the increase in attention sparsity at longer contexts. As detailed in Table 3, the average attention sparsity within the selected layers increases steadily with sequence length for token coverage τ . Together, these results demonstrate that Token Sparse Attention becomes increasingly effective in longer-context regimes, yielding larger efficiency gains when composed with existing attention baselines.

Latency and Overhead Breakdown. Figure 6(b) presents a latency breakdown of Token Sparse Attention at 128K context length. Across all token coverage settings, the additional overhead introduced by Token Sparse Attention remains minimal, accounting for less than 11% of the total attention latency across all layers, even at the highest sparsity level. This overhead includes token scoring and indexing, as well as QKV compression and attention output decompression. These results confirm that Token Sparse Attention achieves significant acceleration while incurring only a small and well bounded overhead, validating its practicality for long context inference.

Dynamic Sparsity vs. Fixed Sparsity. We compare our Dynamic Token Coverage against a fixed token sparsity baseline that keeps a constant fraction of tokens per layer. To ensure a fair comparison, we match the overall attention speedup at 128K by selecting fixed sparsity ratios s that yield similar acceleration to dynamic sparsity. As shown in Table 4, dynamic sparsity consistently achieves higher RULER average accuracy than fixed sparsity under comparable speedups. The gap becomes more pronounced at higher sparsity, where dynamic sparsity preserves accuracy substantially better than fixed sparsity while maintaining comparable acceleration. These results indicate that allocating the sparse budget based on the attention score distribution is more effective than enforcing a rigid token retention ratio, especially in long-context settings where token relevance varies significantly across inputs and layers.

3.4. Comparison with Token Eviction

We compare Token Sparse Attention with representative token eviction methods under a matched efficiency budget. Specifically, we set token coverage to $\tau = 0.01$ and select the hyperparameter settings of PyramidInfer, FastKV and GemFilter that yield similar average attention speedup at 128K. As shown in Table 5, Token Sparse Attention attains the highest average RULER accuracy under comparable

Table 4. Comparison between Dynamic and Fixed Sparsity. τ denotes the token coverage used in dynamic sparsity, while s represents the fixed token sparsity ratio. Sparsity is measured as the average attention map sparsity across the selected sparse layers. Experiments are based on RULER.

Metric	Dynamic Sparsity		Fixed Sparsity	
	$\tau=0.005$	$\tau=0.010$	$s=0.3$	$s=0.5$
Sparsity	54.44%	67.36%	50.96%	74.95%
Accuracy	87.02%	86.84%	86.91%	85.43%
Speedup	$\times 1.36$	$\times 1.51$	$\times 1.32$	$\times 1.57$

speedups. We attribute this advantage to our layer-wise dynamic budget allocation and reversible interleaving, where tokens skipped in one attention operation remain available via the residual path. In addition, head-wise selection enables fine-grained, head-specific token sets, avoiding the rigid unified token constraint of eviction-based methods.

4. Related Works

Long-Context Inference for LLMs. Handling long contexts in large language models introduces significant computational and memory challenges across different inference stages. During the prefill phase, the quadratic complexity of the attention operation leads to substantial computational overhead as the context length increases. To address these issues, system-level approaches such as FlashAttention (Dao et al., 2022; Dao, 2023; Shah et al., 2024) and FlashInfer (Ye et al., 2025) have been proposed to provide efficient attention frameworks. At the same time, algorithmic optimizations for long-context inference are actively being explored.

Prefill Acceleration. One major direction for prefill acceleration is token eviction and prompt compression. Methods such as FastKV (Jo et al., 2025), GemFilter (Shi et al., 2024), and PyramidInfer (Yang et al., 2024a) select important tokens at specific layers during the prefill phase and compress the corresponding hidden states to reduce computation. In contrast, prompt compression methods (Mu et al., 2023; Jiang et al., 2024b; Chuang et al., 2024) operate at the prompt level rather than inside the model, reducing computation by shortening or transforming the input context itself. Other works focus on sparse attention, which skips attention computation in certain regimes based on attention patterns. Minference (Jiang et al., 2024a) applies sparse attention using a vertical-slash pattern, while Flex-Prefill (Lai et al., 2025) introduces query-aware dynamic block-sparse attention. X-Attention (Xu et al., 2025) adopts an antidiagonal scoring strategy to reduce the search cost for identifying sparse patterns, whereas SeerAttention (Gao et al., 2024) trains additional linear layers to search sparse attention patterns. Model compression techniques such as

Table 5. Comparison against Token Eviction. RULER accuracy comparison with token eviction methods on LLaMA-3.1-8B-Instruct. Speedups in the compared methods are set to a similar level, e.g., $\times 1.49$ in PyramidInfer, $\times 1.50$ in FastKV, $\times 1.53$ in GemFilter, and $\times 1.51$ in Ours, compared to FlashAttn.

Method	4K	8K	16K	32K	64K	128K	Avg.
FlashAttn	95.82	92.77	91.02	84.87	83.43	74.15	87.01
PyramidInfer	83.15	79.27	80.74	80.42	77.98	69.35	78.49
GemFilter	92.50	90.87	88.78	85.01	80.42	73.15	85.12
FastKV	94.25	91.30	89.78	84.57	81.54	72.39	85.64
Ours	95.82	92.71	91.46	84.81	83.00	73.25	86.84

quantization are also widely used. These methods reduce memory load time by compressing model weights or further compress activations to enable low-precision computation. Representative approaches include GPTQ (Frantar et al., 2023), SmoothQuant (Xiao et al., 2023), and AWQ (Lin et al., 2024).

Decoding Acceleration. During the decoding phase, the computational bottleneck shifts from quadratic attention computation to the memory and bandwidth cost by KV cache. To address these challenges, methods such as H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), AdaKV (Feng et al., 2024), and HeadKV (Fu et al., 2024) adopt KV cache eviction strategies that retain only important tokens in the KV cache during decoding. In contrast, approaches like ThinK (Xu et al., 2024) reduce decoding overhead by pruning channels of key representations rather than evicting tokens. In addition, applying quantization (Hooper et al., 2024; Liu et al., 2024; Yang et al., 2024b) to keys and values to reduce KV cache loading time has also been actively explored.

5. Conclusion

In this paper, we introduced Token Sparse Attention, a dynamic and reversible token-level sparsification mechanism for efficient long-context inference. The key advantage of Token Sparse Attention lies in its ability to reduce attention computation without permanently removing tokens, allowing token relevance to be re-evaluated across layers and heads. Moreover, our design is fully compatible with existing dense and sparse attention kernels, enabling seamless composition with prior acceleration methods. Experimental results on long-context benchmarks demonstrate that Token Sparse Attention consistently improves the accuracy-latency trade-off across models and tasks. By selectively filtering irrelevant tokens before attention computation, our method achieves substantial attention speedups with minimal accuracy degradation. Ultimately, Token Sparse Attention provides a complementary and practical solution for scalable long-context inference, offering an effective way to improve efficiency while preserving model behavior.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgements

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2025-02273157: Development of Low Power Training/Inference Technologies based on AI Semiconductors, No.RS-2026-25507427: Development of Efficient Architectures and Training Techniques for High-Performance Lightweight AI Models, No. 2021-0-01343: Artificial Intelligence Graduate School Program (Seoul National University)), Samsung Research Funding Center under Project SRFC-TC1603-53, BK21 FOUR program, and the Sejong Fellowship of National Research Foundation of Korea (NRF) funded by the Korea government (MSIT) (No. RS-2025-00561953: Efficient Deep Learning and Inference Algorithms for On-Device AI). (Corresponding Authors: Beomseok Kang and Jae-Joon Kim).

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Anthropic. The claude 3 model family: Opus, sonnet, haiku, 2024.
- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.
- Chuang, Y.-N., Xing, T., Chang, C.-Y., Liu, Z., Chen, X., and Hu, X. Learning to compress prompt in natural language formats. *arXiv preprint arXiv:2402.18700*, 2024.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations (ICLR)*, 2023.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Feng, Y., Lv, J., Cao, Y., Xie, X., and Zhou, S. K. Adakv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, 2024.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations (ICLR)*, 2023.
- Fu, Y., Cai, Z., Asi, A., Xiong, W., Dong, Y., and Xiao, W. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. *arXiv preprint arXiv:2410.19258*, 2024.
- Gao, Y., Zeng, Z., Du, D., Cao, S., Zhou, P., Qi, J., Lai, J., So, H. K.-H., Cao, T., Yang, F., et al. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276*, 2024.
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems*, 37:1270–1303, 2024.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- Jiang, H., Li, Y., Zhang, C., Wu, Q., Luo, X., Ahn, S., Han, Z., Abdi, A. H., Li, D., Lin, C.-Y., et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Advances in Neural Information Processing Systems*, 37:52481–52515, 2024a.
- Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., and Qiu, L. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1658–1677, 2024b.
- Jo, D., Song, J., Kim, Y., and Kim, J.-J. Fastkv: Decoupling of context reduction and kv cache compression for prefill-decoding acceleration. *arXiv preprint arXiv:2502.01068*, 2025.

- Kamradt, G. Needle in a haystack - pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.
- Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. Squeezellm: Dense-and-sparse quantization. *International Conference on Machine Learning (ICML)*, 2024.
- Laban, P., Kryscinski, W., Agarwal, D., Fabbri, A. R., Xiong, C., Joty, S. R., and Wu, C.-S. Summedits: Measuring llm ability at factual reasoning through the lens of summarization. In *Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://api.semanticscholar.org/CorpusID:266164172>.
- Lai, X., Lu, J., Luo, Y., Ma, Y., and Zhou, X. Flexpre-fill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=OfjI1belrT>.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *International Conference on Machine Learning (ICML)*, 2024.
- Mistral AI Team. Mistral-nemo. <https://mistral.ai/news/ministraux>, 2025.
- Mu, J., Li, X., and Goodman, N. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352, 2023.
- Rando, S., Romani, L., Sampieri, A., Franco, L., Yang, J., Kyuragi, Y., Galasso, F., and Hashimoto, T. Long-codebench: Evaluating coding llms at 1m context windows. *arXiv preprint arXiv:2505.07897*, 2025.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:68658–68685, 2024.
- Shi, Z., Ming, Y., Nguyen, X.-P., Liang, Y., and Joty, S. Discovering the gems in early layers: Accelerating long-context llms with 1000x input token reduction. *arXiv preprint arXiv:2409.17422*, 2024.
- Shin, S., Oh, J., and Oh, D. Orthorank: Token selection via sink token orthogonality for efficient llm inference. *arXiv preprint arXiv:2507.03865*, 2025.
- Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pp. 38087–38099. PMLR, 2023.
- Xu, R., Xiao, G., Huang, H., Guo, J., and Han, S. Xattention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428*, 2025.
- Xu, Y., Jie, Z., Dong, H., Wang, L., Lu, X., Zhou, A., Saha, A., Xiong, C., and Sahoo, D. Think: Thinner key cache by query-driven pruning. *arXiv preprint arXiv:2407.21018*, 2024.
- Yang, D., Han, X., Gao, Y., Hu, Y., Zhang, S., and Zhao, H. PyramidInfer: Pyramid KV cache compression for high-throughput LLM inference. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3258–3270, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.195. URL <https://aclanthology.org/2024.findings-acl.195/>.
- Yang, J. Y., Kim, B., Bae, J., Kwon, B., Park, G., Yang, E., Kwon, S. J., and Lee, D. No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization. *arXiv preprint arXiv:2402.18096*, 2024b.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*, 2025.
- Yi, Z., Ouyang, J., Xu, Z., Liu, Y., Liao, T., Luo, H., and Shen, Y. A survey on recent advances in llm-based multi-turn dialogue systems. *arXiv preprint arXiv:2402.18013*, 2024.

Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M., Han, X., Thai, Z., Wang, S., Liu, Z., et al. ∞ -bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15262–15277, 2024.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A. Appendix

A.1. Additional Models and Datasets Details

Models. To evaluate our approach in the long-context regime, we conduct experiments on LLaMA-3.1-8B-Instruct (Dubey et al., 2024) and Mistral-Nemo-12B-Instruct (Mistral AI Team, 2025), designed to operate with 128K context window.

The checkpoints for all models are publicly available at:

LLaMA-3.1-8B-Instruct:

<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>

Mistral-Nemo-12B-Instruct:

<https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407>

RULER. RULER is a configurable synthetic benchmark designed to assess the long-context capabilities of large language models under varying sequence lengths and task difficulties. Rather than focusing on a single retrieval scenario, it generalizes the Needle-in-a-Haystack setup into a comprehensive benchmark suite comprising 13 tasks grouped into four categories: retrieval-style tasks, aggregation tasks (e.g., CWE and FWE), multi-hop tracing tasks (VT), and question answering. This diverse task design allows for controlled and systematic evaluation of a model’s ability to retrieve, aggregate, and reason over long contexts beyond simple keyword matching.

InfiniteBench InfiniteBench is a large-scale benchmark designed to evaluate the ability of language models to understand and reason over extremely long contexts that exceed 100K tokens. It includes a diverse collection of tasks spanning multiple domains, such as retrieval, reasoning, code understanding, mathematical computation, dialogue, and summarization, and covers both synthetic and realistic scenarios.

A.2. Additional Benchmarks

Needle-in-a-Haystack We provide Needle-in-a-Haystack results for LLaMA-3.1-8B-Instruct in Figure 7. Token Sparse Attention demonstrates strong accuracy preservation under long-context settings. When composed with FlexPrefill, Token Sparse Attention consistently achieves higher accuracy than FlexPrefill alone. These results suggest that reversible token-level sparsification effectively removes irrelevant context while preserving the critical signal required for precise retrieval in long-context scenarios.

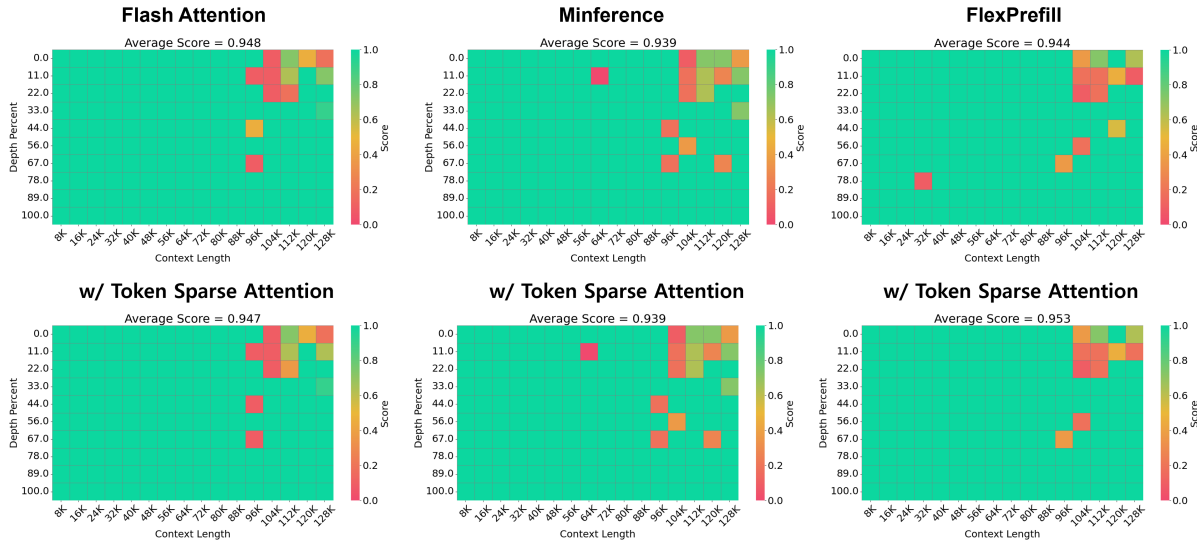


Figure 7. Needle-in-a-Haystack results with Token Sparse Attention on LLaMA-3.1-8B-Instruct.

LongBench. Table 6 reports the LongBench results on LLaMA-3.1-8B-Instruct. LongBench covers a broad spectrum of long-context tasks, including single-document and multi-document question answering, summarization, few-shot learning, synthetic reasoning, and code understanding, providing a comprehensive evaluation of model robustness under long-context settings. Across all baselines, Token Sparse Attention preserves performance consistently while introducing minimal accuracy changes.

Table 6. LongBench results with Token Sparse Attention on LLaMA-3.1-8B-Instruct.

Method	Single-Document QA			Multi-Document QA			Summarization			Few-shot Learning			Synthetic		Code		Avg.
	ArrQA	Qasper	MF-en	HotpotQA	2WikiMQA	MuSiQue	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	LCC	RB-P	PCount	PRe	
Flash-Attention	30.22	45.37	55.80	55.97	45.00	31.26	35.12	25.38	27.20	72.50	91.65	44.32	9.41	99.50	62.90	56.80	49.28
w/ Token Sparse	30.43	45.17	55.34	55.00	45.56	31.28	34.97	25.43	27.09	72.50	91.64	44.07	7.47	98.00	63.06	56.28	48.96
Minference	29.48	45.64	52.59	55.05	42.93	29.24	34.88	24.94	26.53	71.00	91.89	44.15	6.38	98.50	62.38	50.72	47.89
w/ Token Sparse	29.28	46.19	52.52	54.62	42.18	29.89	34.79	25.05	26.48	71.00	91.89	44.07	5.34	94.50	62.79	51.12	47.61
FlexPrefill	27.99	44.63	54.94	57.31	41.82	31.97	34.50	24.92	27.06	70.50	90.74	43.81	4.09	82.00	63.08	60.36	47.48
w/ Token Sparse	28.07	44.84	54.74	56.63	40.24	31.09	34.79	25.47	27.12	69.00	90.89	43.72	3.09	85.50	62.78	60.24	47.39

A.3. Additional Ablation Study

Token Scoring Methods. To validate our scoring design, we conduct additional ablation experiments comparing three alternatives: random query selection, query-only pooling, and our recent-query approach. Random query selection computes importance scores using randomly sampled queries instead of recent queries. Query-only pooling compresses the query tensor by pooling along the sequence dimension and computes approximate attention scores against all keys using the resulting compressed representation.

As shown in Table 7, The results confirm that random query selection leads to noticeable accuracy degradation, and query-only pooling partially recovers accuracy but still falls short of recent-query scoring. These results validate that recent-query scoring is an effective design choice for token-level importance estimation, as it leverages the most contextually relevant queries to identify important tokens without incurring significant computational overhead.

Table 7. Ablation study on scoring methods with Token Sparse Attention on RULER using LLaMA-3.1-8B-Instruct.

Method	Scoring	4K	8K	16K	32K	64K	128K	Avg.
Flash-Attention	–	95.82	92.77	91.02	84.87	83.43	74.15	87.01
w/ Token Sparse	Random Query	94.28	92.49	90.05	82.38	80.83	69.66	84.95
w/ Token Sparse	Only Q-Pooling	95.02	92.55	91.35	84.68	83.04	71.92	86.43
w/ Token Sparse	Recent-Query	96.06	92.90	91.82	84.81	82.83	73.68	87.02

Stability of Drift-Based Layer Selection. To validate the effectiveness of the drift-based layer selection policy introduced in Section 2.4, we conduct an ablation study examining how the choice of sparse layers affects model accuracy. Specifically, we partition all layers into three equal groups based on their normalized drift magnitude (Low, Mid, and High, each comprising the bottom, middle, and top 33% of layers by drift value) and apply Token Sparse Attention exclusively to the layers within each group.

Table 8 reports the mean drift and RULER accuracy for each group across both models. Applying Token Sparse Attention to Low Drift layers consistently yields the highest accuracy, while restricting sparsification to High Drift layers leads to substantial degradation. This trend holds across both LLaMA-3.1-8B-Instruct and Mistral-Nemo-12B-Instruct, demonstrating that drift is a reliable and model-agnostic indicator for identifying layers that can accommodate token-level sparsification with minimal impact on accuracy.

Table 8. RULER Accuracy under drift-based layer grouping for Token Sparse Attention.

Sparse Layer	Mean Drift	4K	8K	16K	32K	64K	128K	Avg.
LLaMA-3.1-8B-Instruct								
Low Drift	0.204	95.82	92.52	90.95	84.87	83.19	73.99	86.89
Mid Drift	0.348	95.38	92.28	91.40	83.32	81.21	69.48	85.51
High Drift	0.553	94.67	91.74	89.08	68.88	50.24	38.17	72.13
Mistral-Nemo-12B-Instruct								
Low Drift	0.088	95.13	92.22	85.30	64.26	46.71	19.30	67.15
Mid Drift	0.146	95.17	91.74	81.09	66.73	42.45	18.64	65.97
High Drift	0.299	70.92	29.57	24.93	17.43	6.41	4.03	25.55

Delta Sweep for Sparse Layer Selection. Table 9 reports the accuracy-speedup trade-off across varying delta values on RULER for LLaMA-3.1-8B-Instruct. The layer selection threshold delta controls the proportion of layers eligible for token-level sparsification, where a larger delta includes more layers with higher representation drift. The results demonstrate that $\delta=0.5$ serves as a consistent sweet spot, balancing accuracy preservation and speedup. Values beyond 0.5 yield only marginal additional speedup while incurring notable accuracy degradation.

Table 9. Effect of delta on Token Sparse Attention on RULER using LLaMA-3.1-8B-Instruct.

Method	δ	4K	8K	16K	32K	64K	128K	Avg.	Speedup
Flash-Attention	–	95.82	92.77	91.02	84.87	83.43	74.15	87.01	$\times 1.00$
w/ Token Sparse	0.1	95.82	92.47	91.16	84.95	83.02	73.94	86.89	$\times 1.12$
w/ Token Sparse	0.3	95.82	92.71	91.00	84.87	83.19	73.91	86.92	$\times 1.24$
w/ Token Sparse	0.5	96.06	92.90	91.82	84.81	82.83	73.68	87.02	$\times 1.36$
w/ Token Sparse	0.7	95.30	92.62	90.58	83.44	81.53	68.63	85.35	$\times 1.46$
w/ Token Sparse	0.9	94.68	89.94	90.23	73.14	70.55	56.41	79.16	$\times 1.53$

A.4. Prefill Latency

Our method and baselines target prefill acceleration, and since all methods use dense attention with full KV cache during decoding, TPOT is identical across all methods (89ms at 128K with 128 generation tokens, measured on FlashAttention). End-to-end gains are therefore captured through prefill latency. To further support this, we additionally report full prefill latency (TTFT) on both A100 and A6000 GPUs.

As shown in Table 10 and Table 11, at 128K, prefill latency improvements closely follow attention-level speedups on both hardware platforms, confirming that attention dominates in long-context regimes. At 8K, the attention sparsity decreases as shown in Table 3, and the scoring overhead becomes relatively more visible. However, the overhead introduced by Token Sparse Attention remains small compared to other baselines, and our primary goal is to push the efficiency frontier in long-context inference, where consistent gains are observed across all methods and both hardware configurations.

Table 10. Prefill latency comparison across 8K and 128K sequence lengths on a single A100 GPU

Method	8K		128K	
	Latency (sec)	Speedup	Latency (sec)	Speedup
Flash-Attention	0.68 ± 0.0061	$\times 1.00$	31.04 ± 0.0569	$\times 1.00$
w/ Token Sparse	0.70 ± 0.0064	$\times 0.97$	24.35 ± 0.0195	$\times 1.27$
Minference	1.52 ± 0.0079	$\times 0.45$	26.84 ± 0.0492	$\times 1.16$
w/ Token Sparse	1.53 ± 0.0103	$\times 0.44$	23.32 ± 0.0124	$\times 1.33$
FlexPrefill	0.90 ± 0.0084	$\times 0.76$	18.45 ± 0.0439	$\times 1.68$
w/ Token Sparse	0.92 ± 0.0098	$\times 0.74$	16.81 ± 0.0272	$\times 1.85$

Table 11. Prefill latency comparison across 8K and 128K sequence lengths on a single A6000 GPU

Method	8K		128K	
	Latency (sec)	Speedup	Latency (sec)	Speedup
Flash-Attention	1.76 ± 0.3624	$\times 1.00$	67.34 ± 0.4273	$\times 1.00$
w/ <i>Token Sparse</i>	2.04 ± 0.2284	$\times 0.86$	54.49 ± 0.1430	$\times 1.24$
Minference	4.00 ± 0.5044	$\times 0.44$	53.54 ± 0.6051	$\times 1.26$
w/ <i>Token Sparse</i>	3.71 ± 0.4937	$\times 0.47$	47.05 ± 0.3972	$\times 1.43$
FlexPrefill	3.22 ± 0.5334	$\times 0.55$	47.16 ± 0.3978	$\times 1.43$
w/ <i>Token Sparse</i>	2.68 ± 0.3400	$\times 0.66$	43.08 ± 0.4519	$\times 1.56$

A.5. Comparison with Additional Baselines

To further validate the complementary nature of Token Sparse Attention, we conduct additional experiments composing it with SeerAttention (Gao et al., 2024) and X-Attention (Xu et al., 2025) on LLaMA-3.1-8B-Instruct. Both methods apply structured sparsity patterns at the attention map level, and Token Sparse Attention operates orthogonally at the Q, K, V tensor level, enabling seamless composition without modifying the underlying sparse kernels.

As shown in Table 12, composing Token Sparse Attention with SeerAttention improves the attention speedup from $\times 2.19$ to $\times 2.47$ while maintaining comparable accuracy. Similarly, when composed with X-Attention ($S=16$, $\tau=0.9$), the speedup increases from $\times 2.72$ to $\times 3.49$ with negligible accuracy change. These results are consistent with the complementary gains observed with Minference and FlexPrefill in the main experiments, and suggest that Token Sparse Attention provides a general acceleration benefit that extends across diverse sparse attention kernels regardless of their specific sparsity pattern.

Table 12. Complementary Gains of additional methods from Token Sparse Attention on RULER.

Method	4K	8K	16K	32K	64K	128K	Avg.	Speedup
LLaMA-3.1-8B-Instruct								
Flash-Attention	95.82	92.77	91.02	84.87	83.43	74.15	87.01	$\times 1.00$
w/ <i>Token Sparse</i>	96.06	92.90	91.82	84.81	82.83	73.68	87.02	$\times 1.36$
SeerAttention	95.82	92.75	90.97	84.81	83.71	73.85	86.99	$\times 2.19$
w/ <i>Token Sparse</i>	95.82	92.84	91.47	85.17	83.19	73.60	87.02	$\times 2.47$
X-Attention	96.14	92.46	89.99	84.05	76.05	61.65	83.39	$\times 2.72$
w/ <i>Token Sparse</i>	96.14	92.16	90.01	84.26	76.03	62.43	83.51	$\times 3.49$

A.6. Comparison with Non-Eviction Method

We compare Token Sparse Attention against OrthoRank (Shin et al., 2025), a non-eviction method that identifies important tokens based on their orthogonality to the sink token in the normalized hidden state space. Although OrthoRank avoids permanent token removal, we note two fundamental architectural differences that distinguish it from Token Sparse Attention.

Compression Level and Scoring Mechanism. OrthoRank operates at the layer input hidden state level, selecting tokens prior to the attention module based on a geometric metric derived from the normalized hidden state space. In contrast, Token Sparse Attention performs compression at the Q, K, V level within each attention head, using the attention map itself as the scoring signal. Since our goal is to identify which tokens are important specifically for the attention computation, using the attention map as a scoring proxy directly captures the degree to which each token contributes to the attention output at the current layer and head.

Per-Head Independent Token Selection. OrthoRank selects tokens based on the hidden state prior to the attention module, which means all attention heads within the same layer share an identical token set. This design is unable to account for the head-wise divergence in attention patterns. As shown in Figure 2(b), different attention heads within the same layer exhibit significantly different token importance rankings, and enforcing a single unified token set across all heads inevitably discards tokens that certain heads would find essential.

To empirically validate these differences, we reproduced OrthoRank and extended it to the long-context evaluation setting. For a fair comparison, we use prefill speedup rather than attention speedup as the efficiency metric, since OrthoRank additionally skips FFN computation for unselected tokens, making attention speedup alone an insufficient basis for comparison.

As shown in Table 13, Token Sparse Attention achieves substantially higher average RULER accuracy (87.02% and 86.84%) compared to OrthoRank at both pruning ratios (79.36 % and 78.38%) under comparable prefill speedup. The accuracy gap is particularly pronounced at longer context lengths, with a difference of approximately 17% at the 128K setting. We attribute this gap primarily to the two architectural differences described above.

Table 13. RULER accuracy comparison with non-eviction method on LLaMA-3.1-8B-Instruct. p denotes the pruning ratio.

Method	4K	8K	16K	32K	64K	128K	Avg.	Prefill Speedup
LLaMA-3.1-8B-Instruct								
Flash-Attention	95.82	92.77	91.02	84.87	83.43	74.15	87.01	$\times 1.00$
OrthoRank ($p=0.30$)	90.25	89.85	86.37	81.10	72.60	55.98	79.36	$\times 1.27$
OrthoRank ($p=0.35$)	90.86	86.23	84.54	80.89	71.36	56.39	78.38	$\times 1.31$
Token Sparse ($\tau=0.005$)	96.06	92.90	91.82	84.81	82.83	73.68	87.02	$\times 1.27$
Token Sparse ($\tau=0.010$)	95.82	92.71	91.46	84.81	83.00	73.25	86.84	$\times 1.32$

A.7. Limitation

While Token Sparse Attention is primarily designed for long-context inference, its efficiency gains are naturally smaller at short context lengths where attention computation does not yet dominate end-to-end latency. This behavior is common to most coverage-based sparse attention methods and does not detract from the effectiveness of our approach in the long-context regimes that are the main focus of this work. Meanwhile, applying overly aggressive token sparsification can increase the risk of excluding semantically important tokens from attention, potentially leading to accuracy degradation. However, Token Sparse Attention is designed to remove irrelevant tokens that contribute primarily to attention noise rather than informative context. By selecting an appropriate token coverage, the method enables a controlled and practical accuracy–efficiency trade-off, which is the primary objective in long-context regimes. Finally, while this work focuses on prefill acceleration for text generation models, Token Sparse Attention can be extended in several promising directions. Future work includes adapting the proposed mechanism to the decoding phase, as well as exploring its applicability to multimodal settings such as vision–language models.